

CS247 Assignment 4 GPU Volume Raycasting

How to run

```
brew install glew          # one-time
cd Assignment_4
cmake -S . -B build
cmake --build build -j
cd build && ./assignment
```

Press 1 for lobster or 2 for head, then f to switch DVR / iso-surface.

Key	Action
f	DVR / iso-surface
l	MIP (bonus)
5-0	pick TF0 to TF5
i / k	iso value ± 0.05
w/s, e/d	move TF window lower / upper bound
+ / -	step size
o	toggle shading, g edge boost
left-drag	rotate, Home reset, Esc quit

Implementation

No geometry extraction. Each frame I render the cube's front and back faces into two FBOs (RGB = tex coord), then a full-screen quad's fragment shader walks each ray from entry to exit and samples the 3D volume. Two modes: DVR (front-to-back compositing through a transfer function) and iso-surface (stop at the first iso crossing). Most of the host code was given; my work is the two GLSL shaders plus the transfer-function code.

Ray setup (both shaders).

```
vec3 rayStart = texture2D(frontfaces, uv).xyz;
vec3 rayEnd   = texture2D(backfaces, uv).xyz;
vec3 dir      = normalize(rayEnd - rayStart);
int steps    = int(length(rayEnd - rayStart) / step_size) + 2;
```

Pixels where the ray length is ~ 0 (outside the cube) are discarded.

Iso-surface (raycast_iso.glsl). March the ray; on a sign change of (sample - iso) refine the hit $t = (\text{iso-prev}) / (v\text{-prev})$, compute a central-difference normal, do Lambertian shading, and return immediately (early ray termination). No crossing means discard.

DVR (raycast_dvr.glsl). Front-to-back compositing:

```
src.a = 1.0 - pow(1.0 - src.a, step_size / REF_STEP); // opacity correction
...
accum.rgb += (1.0 - accum.a) * src.a * src.rgb;
accum.a   += (1.0 - accum.a) * src.a;
if (accum.a > 0.95) break; // early ray termination
```

Opacity correction (Beer-Lambert) keeps the density consistent when the step size changes. Optional per-sample shading uses a central-difference gradient, with an edge boost on g .

Transfer function (UpdateTransferfunction). A 128-entry 1D texture mapping intensity to RGBA. TF0 is a windowing TF: linear ramp inside `[win_min, win_max]`, transparent outside. The w/s/e/d keys move the bounds and re-upload, so narrowing the window isolates an intensity range.

MIP (bonus). Toggled by l: take the max value along the ray and look it up through the TF.

Results

See `demo.mov`: DVR with different TFs, shading on/off, iso-surface at different iso values, and MIP. Pre-integrated TF / empty-space skipping / clipping planes not done.